

Working With Code Samples

Contents

- [Compiling the code samples](#)
- [Running the program](#)
- [Debugging](#)
- [Moving From Simulation To Hardware](#)

The [CSL Code Examples](#) section contains CSL programs, the `.csl` files, that each either demonstrate individual features of the language, or solve a larger application problem. Each program is accompanied by a Python script, the `run.py` file, to run that program with the simulator.

The source for these code samples can be found inside the `csl-extras-{build id}.tar.gz` tarball within the release, or at the [SDK examples GitHub repository](#) (request access from developer@cerebras.net).

For the GEMV tutorial code samples, we additionally provide step-by-step explanations of the code in the [Tutorials](#) section.

Warning

If you're just getting started, we recommend walking through the step-by-step tutorials in [Tutorials](#) to get a fuller explanation of these programs.

Compiling the code samples

Each code sample contains a CSL file as the top-level source file, typically named `layout.csl`. This file may reference additional CSL source files in that directory.

Each code sample also contains a `commands.sh` script, which contains the commands required to compile and run the example.

For example, the `tutorials/gemv-01-complete-program/commands.sh` at [GEMV 1: A Complete Program](#) contains:

```
cslc ./layout.csl --fabric-dims=8,3 \  
  --fabric-offsets=4,1 -o out --memcpy --channels 1  
cs_python run.py --name out
```

See also

See [CSL Compiler](#) for the compiler options documentation.

To compile the program:

1. First cd into the directory that contains the CSL files.
2. Then run the `cslc` command shown in the `commands.sh` file to compile the program. Note, this command may span multiple lines. This command execution will produce files with the `elf` extension. For example:

```
$ cd tutorials/gemv-01-complete-program/  
$ cslc ./layout.csl --fabric-dims=8,3 --fabric-offsets=4,1 -o out --memcpy --channels 1  
$ ls out  
bin east generated out.json west  
$ ls out/bin  
out_0_0.elf out_rpc.json
```

Running the program

Use the `run.py` Python script that is in the code sample directory to execute the compiled program. For example, to run the above compiled program, execute the following command in the `gemv-01-complete-program` directory:

```
$ cs_python run.py --name out
```

If the program runs correctly, you will see the message `SUCCESS!` near the end of the output.

Debugging

See [Debugging Guide](#) and [SDK GUI](#).

Moving From Simulation To Hardware

After successfully simulating your CSL program, you can run it on hardware by following the below guidelines when using `cslc`:

Pass `--arch` flag

Use the `--arch` flag with `cslc`. This will ensure that the appropriate Cerebras system is targeted.

Allowed values are `--arch=wse2` for WSE-2 architecture and `--arch=wse3` for WSE-3 architecture. The default value is `wse2`. For example:

```
cslc --arch=wse2 ./layout.csl --fabric-dims=8,3 \  
--fabric-offsets=4,1 -o out --memcpy --channels 1
```

Note that `wse3` is not yet supported for all example programs.

Provide `--fabric-dims`

When compiling for simulation with `cslc`, the `--fabric-dims` flag can be any bounding box large enough to contain your program's PEs. However, when compiling for hardware, these dimensions must match your Cerebras system's fabric dimensions. For example:

```
cslc --arch=wse2 ./layout.csl --fabric-dims=757,996 \  
--fabric-offsets=4,1 -o out --memcpy --channels 1
```

Provide IP address to `SdkRuntime`

To run on the Cerebras system hardware, you must pass the IP and port address of the network-attached Cerebras system to the `cmaddr` argument of the `SdkRuntime` constructor in your `run.py`:

```
runner = SdkRuntime(compile_dir, cmaddr="1.2.3.4:9000")
```